

SimTwin: an open-source reproducible simulator of networked FDM/FFF printer fleets with digital-twin synchronisation, fault injection and communication impairment

Nata Sulakvelidze^{a,*}

^a*Versec LLC / G3D, Tbilisi, Georgia*

Abstract

Digital-twin research on additive-manufacturing (AM) fleets needs experiments in which the physical process, the communication channel and the twin’s synchronisation state can all be manipulated and measured independently. Existing tools cover fragments of this: discrete-event factory simulators ignore the physics of the print, thermal and process models of a single machine ignore the network, and networked-control benchmarks have no plant. We present SimTwin, an open-source Python simulator that couples (i) a three-node lumped-parameter thermal plant with proportional–integral heater control, an additive power model and a road-geometry deposition model, (ii) a finite-state job and health machine per device, (iii) an explicit message-level transport with latency, jitter, loss, duplication, per-device disconnection, broker-wide outage, replay queues and staleness-based discarding, and (iv) a twin registry that maintains *reported*, *desired*, *configuration* and *connectivity* aspects separately and is written only by delivered messages. Faults are injected as *mechanisms* — multiplicative changes to heater efficiency, thermal conductance, parasitic heat load, controller gain, extrusion scale, motor power, sensor bias and sensor noise, plus terminal job interruption — never as scripted telemetry labels, so fault signatures emerge from the dynamics. A seed-spawning scheme gives bit-level reproducibility: two runs at the same seed produce byte-identical telemetry (SHA-256 match),

*Corresponding author.

Email address: `nata.sulakvelidze@gmail.com` (Nata Sulakvelidze)

and a change of seed changes it. We report seven experiments on a single 11-core laptop: scalability to 1 000 printers at $88\,643\text{ device-steps s}^{-1}$ and a real-time factor of 88.6; telemetry throughput from 686 to $70\,363\text{ msg s}^{-1}$ over a 240-fold sampling sweep; delivery ratios that track the configured loss rate to within 0.05 percentage points while 95th-percentile twin staleness grows from 0.99 s to 6.99 s; outage sweeps in which resynchronisations grow from 0 to 324 942; and fault separability in which 10 of 11 injected fault families reach Cohen’s $|d| \geq 1.45$ on at least one observable channel, with bed-heater failure at $|d| = 3.68$ on measured power; and a broker-fidelity study in which the same messages traverse a real Mosquitto broker, showing that QoS 1 back-pressure holds 95th-percentile twin staleness at 1.2 s while fire-and-forget QoS 0 lets it grow 12.6-fold to 15.1 s. Together these give a measurable, citable baseline for twin-synchronisation quality of service and for mechanism-grounded fault-signature separability in networked AM fleets.

Keywords: digital twin, additive manufacturing, fused deposition modeling, fleet simulation, fault injection, industrial IoT, MQTT, age of information, reproducibility

1. Introduction

§2 positions the work, §4 gives the model and §3 the implementation, §4.8 the reproducibility contract, §6 the seven experiments, and §8–§9 what they mean and do not mean.

Fused deposition modelling (FDM/FFF) fleets are becoming a networked, instrumented production setting: tens to hundreds of machines report telemetry over MQTT or OPC UA to a supervisory layer that maintains a digital twin of each machine (Grieves and Vickers, 2016; Tao et al., 2019; Barricelli et al., 2019; Fuller et al., 2020). The digital-twin literature is explicit that the value of a twin depends on the *fidelity* of the model, the *freshness* of its state, and the *fidelity* of the observed data (Tao et al., 2017; International Organization for Standardization, 2021, 2023), and that synchronisation is itself an engineering problem with a cost (Tan and Matta, 2023).

Research on this setting needs an experiment platform in which three things can be varied and measured *independently*: the physical process, the communication channel, and the twin’s synchronisation state. The tools that exist each cover one axis and assume the others away. Discrete-event factory simulators model queues and stations, not nozzle thermodynamics.

Thermal and process models of a single extruder (Faria et al., 2020) model the physics in detail, for one machine, over an ideal channel. Networked-control-system studies model delay, dropout and scheduling (Walsh et al., 2002; Zhang et al., 2001) over an abstract plant. Age-of-information (AoI) work quantifies freshness of a source (Kaul et al., 2012; Yates et al., 2021) but has no process to be fresh *about*. AM fault-detection datasets are dominated by images and single-machine instrumented runs (Szydło et al., 2021); predictive-maintenance benchmarks use machine-tool signals without a fleet, a network, or a twin.

This paper contributes SimTwin, a simulator built to close that gap, and the reproducible evidence base that comes with it.

Contributions

1. **A layered architecture with a hard information barrier.** Device state, twin-reported state, desired state, configuration and connectivity are separate objects. The twin registry is written *only* by messages the transport delivered, so twin staleness, synchronisation delay and resynchronisation counts are consequences of the channel model rather than annotations.
2. **Mechanism-level fault injection.** Eleven faults across five families are expressed as parameter transformations of the plant and observation models (Table 1). Fault signatures are emergent, which is what makes a separability study meaningful.
3. **Explicit delivery semantics.** Latency with a lognormal tail, independent loss, duplication, per-device disconnection with exponential reconnect backoff and replay queues, broker-wide outage, and staleness-based discarding, all applied in the transport, never in the device model.
4. **Determinism by construction.** A `SeedPlan` spawns independent named NumPy streams, so a run is a pure function of its configuration and seed. We verify this by hashing telemetry.
5. **Seven experiments with published artifacts** covering scalability, telemetry frequency, network impairment, fault separability, outage/recovery, reproducibility and broker fidelity, plus a dataset package with provenance metadata.
6. **A fully verified bibliography:** every journal, volume, issue, page range and DOI in this paper was resolved against the Crossref REST

API; five candidate sources that could not be resolved were dropped (Section 8).

SimTwin is deliberately *not* a high-fidelity FDM physics simulator. It is a cyber-physical / industrial-IoT research testbed: the physics is the simplest lumped model that produces the qualitative behaviour the twin has to track, at a cost that lets a thousand-machine, hour-long scenario run on a laptop in under a minute.

2. Related work

2.1. *Digital twins in manufacturing*

Grieves’ twin formulation separates the physical asset, its virtual model and the connections between them (Grieves and Vickers, 2014, 2016). Tao et al. formalise a five-layer architecture and the requirement for bidirectional coupling (Tao et al., 2019, 2017), and survey twin-driven smart-manufacturing system design (Liu et al., 2023). Standardisation has caught up: ISO 23247 defines a digital-twin framework for manufacturing (International Organization for Standardization, 2021) and ISO/IEC 30173 defines digital-twin representations of physical systems (International Organization for Standardization, 2023). Surveys map the enabling-technology landscape (Fuller et al., 2020; Barricelli et al., 2019), the maturity models used to classify twins (Uhlenkamp et al., 2022), and the methodologies by which twins are built (Aznar Lapuente et al., 2026). Reference architectures for synchronised twin factories are demonstrated in (Cho and Noh, 2024), and simulation-based twins for discrete material-flow systems in (Schwede and Fischer, 2024). The *synchronisation problem* — how far a twin may diverge from its physical twin, how that divergence is formulated, and what it costs to close the gap — is treated explicitly by Tan and Matta (2023). SimTwin is built to make that divergence measurable: it exposes per-twin age of information, synchronisation-delay percentiles, staleness and offline areas, and resynchronisation counts as first-class outputs.

2.2. *AM/FDM digital twins and process simulation*

Twin-AM integration is an active review area (Jyeniskhan et al., 2024). Machine-level twins have been demonstrated for FDM failure detection (Henson et al., 2021), for performance monitoring and anomaly detection in industry (Alhazmi et al., 2024), as a real-time twin ecosystem for AM (Pantelidakis et al., 2022), and as open-source reference implementations (Robles

et al., 2023; Infante et al., 2025; Larsen et al., 2024). FIBR3DEmul is the closest physical analogue to SimTwin’s process layer: a multi-axis extrusion-deposition simulator for FDM (Faria et al., 2020). It targets bead geometry and material deposition for one machine; SimTwin targets fleet-level telemetry, connectivity and twin state, and uses a deliberately coarser deposition model (Section 4.3). Energy and resource accounting for AM is well established (Baumers et al., 2017; Wippermann et al., 2020); the SimTwin power model is an additive subsystem model in that tradition, calibrated to the order of magnitude of desktop FDM machines rather than fitted to a specific machine.

2.3. IIoT transport, MQTT and networked control

MQTT (OASIS Standard, 2014, 2019) and OPC UA (Cavalieri and Cutuli, 2010) dominate IIoT messaging; publish/subscribe protocols for industrial settings are compared by Nast et al. (2022). Broker performance under load has been measured repeatedly (Mishra and Kertesz, 2020; Mishra et al., 2021; Silva et al., 2021), including CoAP/MQTT comparisons (Seoane et al., 2021), security-aware comparisons (Lakshminarayana et al., 2024), and MQTT-specific intrusion datasets (Alatram et al., 2023). MQTT in a twin data pipeline is demonstrated by Human et al. (2021), and the standards landscape for twin-enabled IoT by Jacoby and Usländer (2020). Networked control has a long literature on the control consequences of sampling, delay and dropout (Zhang et al., 2001; Walsh et al., 2002). Age-of-information gives the queueing-theoretic language for freshness (Kaul et al., 2012; Yates et al., 2021). SimTwin adopts the AoI view for the twin side and the dropout/delay view for the channel side, and puts both on top of a plant that produces the data.

2.4. Fault detection, anomaly detection and simulation reporting

Model-based fault detection and diagnosis is the classical framework for residual generation from a physical model (Isermann, 2005). Data-driven anomaly detection supplies the statistics used to score fault separability here: local outlier factor (Breunig et al., 2000; Blázquez-García et al., 2021), isolation forests (Liu et al., 2008), one-class SVMs (Schölkopf et al., 2001), and deep approaches on AM process signals (Zamanzadeh Darban et al., 2024). Synthetic data is an established substitute for failure data that fleets do not generate in sufficient quantity (Nieminen et al., 2026); the published

faulted-fleet dataset in Section 7 is offered in that spirit. Reporting guidelines for simulation-based research motivate the explicit treatment of seeds, environment capture and artifact publication in Section 7 (Rahmandad and Sterman, 2012).

2.5. *Position of SimTwin*

No prior open-source tool that we could verify combines (a) a per-device physical plant, (b) a finite-state job and health machine, (c) message-level network impairment with delivery semantics, (d) a twin registry whose state is derived only from delivered messages, and (e) mechanism-level fault injection with a published separability baseline. That combination, not the fidelity of any one layer, is the contribution.

3. System architecture

The architecture realises the model of §4 as four cooperating parts: a vectorised plant, a twin registry, a transport, and a fault injector.

SimTwin is a Python 3 package (`simtwin`) organised as nine modules. The fleet engine (`fleet/engine.py`) is the integrator; everything else is a model it composes.

- `config.py` — thirteen dataclass configuration groups (experiment, fleet, simulation, thermal, power, material, job, health, network, twin, fault, storage, MQTT) with a `validate()` pass that rejects physically inconsistent parameter combinations before a run starts.
- `printer/thermal.py` — three coupled first-order thermal nodes (nozzle, bed, chamber) with PI heater control (Section 4.2).
- `printer/process.py` — additive subsystem power model, road-geometry deposition model, stochastic job generator (Section 4.3).
- `printer/states.py` — the device finite-state machine (Section 4.4).
- `faults/catalogue.py`, `faults/injector.py` — the fault catalogue and the scheduler that turns it into per-device parameter fields (Section 4.5).
- `communication/message.py`, `communication/transport.py` — the message model and the delivery-semantics pipeline (Section 4.6).

- `twin/registry.py` — the twin registry and synchronisation accounting (Section 4.7).
- `storage/writers.py` — CSV, JSONL and Parquet writers with environment and configuration provenance.
- `metrics/quality.py` — resource sampling and the separability statistics (Cohen’s d , Mann–Whitney AUC).
- `utilities/seeds.py`, `utilities/environment.py` — deterministic stream spawning and runtime environment capture.

The engine is *vectorised*: the fleet is a set of NumPy state arrays of length n , and one simulated timestep advances all devices with array operations. This is what makes a 1 000-machine, hour-long scenario a 51-second job (Section 6.1).

4. Model

4.1. Time base

Time advances in steps of Δt seconds (default 1 s). Telemetry is published every T_{tel} milliseconds (default 1 000 ms), and the transport is stepped every step, so delivery latencies finer than Δt are represented by a continuous delivery-time distribution while the device state is sampled at the step grid. Configuration validation enforces $T_{\text{tel}} \geq 1000 \Delta t$; experiments that need sub-second telemetry lower Δt so that every interval is an integer multiple of the step (Section 6.2).

4.2. Thermal plant

Each device has three lumped thermal nodes $i \in \{n, b, c\}$ (nozzle, bed, chamber) with capacitance C_i [J/K] and conductance to the ambient sink G_i [W/K]:

$$C_i \frac{dT_i}{dt} = P_i^{\text{heat}} - G_i (T_i - T_{\text{amb}}) + \kappa_i Q_i^{\text{coup}} - \Lambda_i + \xi_i(t), \quad (1)$$

where $P_i^{\text{heat}} = u_i P_i^{\text{max}}$ with duty $u_i \in [0, 1]$, Λ_i is a parasitic heat load, κ_i couples dissipation from the hot nodes into the chamber, and $\xi_i(t)$ is an Ornstein–Uhlenbeck process with correlation time τ_ξ and stationary standard

deviation σ_ξ —coloured, not white, process noise. Heaters are controlled by a saturating proportional–integral law on the *measured* temperature,

$$u_i(t) = \text{clip}\left(K_p e_i(t) + \frac{K_p}{T_i^{\text{int}}} \int_0^t e_i(s) ds, 0, 1\right), \quad e_i = T_i^{\text{set}} - \tilde{T}_i, \quad (2)$$

where $\tilde{T}_i$ is the sensor reading, so sensor bias and sensor noise enter the control loop exactly as they do on a real machine. Discretising (1) with step Δt gives the update used by the engine, and (2) is integrated as a running sum per heater. The open-loop time constants at the defaults of Table 2 are $\tau_n = C_n/G_n = 200$ s, $\tau_b = 400$ s and $\tau_c = 1800$ s, and the full-power steady-state rises over ambient are $P_n^{\text{max}}/G_n = 525$ K, $P_b^{\text{max}}/G_b = 83$ K and $P_c^{\text{max}}/G_c = 100$ K, which are the orders that make the setpoints 210/60/45 °C attainable with duty cycles in the interior of $[0, 1]$. The chamber heater is disabled by default: an open-frame desktop machine has no chamber heater, and the chamber warms by coupling.

4.3. Process, power, material and jobs

Total electrical draw is an additive subsystem model,

$$P = P^{\text{elec}} + P_n^{\text{heat}} + P_b^{\text{heat}} + P_c^{\text{heat}} + P_{\text{hot}}^{\text{fan}} + P_{\text{part}}^{\text{fan}} + \left(P_0^{\text{mot}} + P_v^{\text{mot}} \frac{v}{v_{\text{nom}}}\right) + P^{\text{ext}} + \epsilon_P, \quad (3)$$

with $P^{\text{elec}} = 8$ W standby, hot-end and part fans, a motion term scaling with normalised speed, and an extruder term; (3) is the quantity accumulated as energy use and compared against the nameplate rating in E4. Deposited volume follows the road geometry, $\dot{V} = w h v$ for line width w , layer height h and print speed v ; mass follows from filament density, and material use integrates $\dot{V}$ over the print. Jobs are drawn from a generator with duration $\mathcal{U}[1800, 21600]$ s, layer count $\mathcal{U}[60, 900]$, part volume $\mathcal{U}[5 \times 10^3, 2.2 \times 10^5]$ mm³ and speed $\mathcal{U}[40, 120]$ mm/s, with 5 % relative jitter on realised duration. Health accumulates wear at 10^{-6} health-points per operating second, multiplied by the active fault’s wear multiplier; preventive maintenance is scheduled below a health of 0.35 and restores 0.92.

4.4. Device state machine

Each device runs a finite-state machine over OFFLINE, IDLE, PREHEAT, PRINTING, PAUSED, COOLDOWN, FAILED and MAINTENANCE. Preheat completes

Table 1: Fault catalogue. Coefficients are the values reached at full severity. *Term.* marks a fault that aborts the active job; *Obs.* marks a fault that changes only the observation model.

Fault	Family	Mechanism at full severity	Term.	Obs.	Wear
nozzle_heater_failure	thermal	nozzle heater efficiency $\rightarrow 0$			3.0
bed_heater_failure	thermal	bed heater efficiency $\rightarrow 0$			2.0
temperature_instability	thermal	nozzle proportional gain $\times 6$ (sustained limit cycle)			1.5
nozzle_thermal_drift	thermal	+8 W parasitic nozzle heat load			2.0
abnormal_cooling	thermal	nozzle conductance $\times 4$			1.5
extrusion_blockage	extrusion	deposition $\times 0.4$, extruder power $\times 1.8$			4.0
motor_overheating	mechanical	motion power $\times 2.5$			3.0
power_anomaly	electrical	total power $\times 1.35$			2.0
sensor_drift_nozzle	observat.	+15 K additive nozzle bias		✓	1.0
sensor_noise	observat.	nozzle/bed noise std. $\times 10$		✓	1.0
print_interruption	mechanical	unrecoverable controller fault	✓		5.0

when all enabled nodes are within 3 K of setpoint for at least 20 s, and aborts as **FAILED** after 900 s. A terminal fault moves **PRINTING** $\rightarrow$ **FAILED** $\rightarrow$ **MAINTENANCE**, with repair time $\text{Exp}(1/1800\text{s})$. The state machine is evaluated on the device side; the twin’s `reported_state` is a copy that arrives in a message.

4.5. Fault mechanisms

This is the part of the simulator that E4 (§6.4) exercises directly. Faults are *not* telemetry labels. Each fault is a set of coefficients applied to the plant and observation models, ramped linearly from nominal to full severity over the fault’s ramp-in and back over its ramp-out, with severity drawn from the fault’s severity range. Table 1 lists the catalogue. Two faults are marked *observational*: they change only what the twin can observe, not the plant, which is what lets the evaluation distinguish a physical fault from a sensing fault. Faults are scheduled by a Poisson process at a configured rate per machine-hour, with per-family weights, and each injection is logged as an event with device, time, severity and duration.

4.6. Transport and delivery semantics

Every publication carries a generation timestamp, a kind (**TELEMETRY**, **HEALTH**, **ALERTS**, **STATE**, **COMMAND**, **CONFIGURATION**, **LWT**), a QoS level and a device id. The transport pipeline applies, in order:

1. **Connectivity.** Per-device disconnection episodes with mean interval $\lambda_{\text{dev}}^{-1}$ and mean duration μ_{dev}^{-1} , with exponential reconnect backoff 2^k s at the k -th failed attempt; and broker-wide outage episodes.
2. **Latency.** $L = \max(0, L_0 + \mathcal{N}(0, \sigma_j))$ plus a lognormal tail component with shape σ_{ln} , so the bulk of the distribution is tight and the tail is heavy.
3. **Loss.** Independent per-message Bernoulli loss with probability p (QoS 0 semantics).
4. **Duplication.** With probability p_{dup} a QoS ≥ 1 message is delivered twice (at-least-once semantics).
5. **Replay.** While a device is disconnected, its messages enter a bounded FIFO replay queue (limit 10^4). Messages older than $\tau_{\text{age}} = 300$ s at delivery are dropped as stale. Delivered replayed messages are flagged **replayed**.

All of this happens in the transport. The device model never learns that a message was lost, which is the point: a fault in the plant and a fault in the channel are separately attributable.

4.7. Twin registry and synchronisation metrics

Each twin holds the reported state, the desired state, the confirmed configuration, the connectivity state, health, fault code, current job, progress, temperatures, power and material use, and the bookkeeping fields `t_last_accepted`, `updates_accepted` and `resync_count`. The *only* write path is `apply(msg)`, called on delivery. From this we define, at simulated time t :

$$\text{staleness}(t) = t - t_{\text{last_accepted}}, \quad (4)$$

$$\text{status}(t) = \begin{cases} \text{OFFLINE}, & \text{staleness} > \tau_{\text{off}} \text{ or last will,} \\ \text{STALE}, & \tau_{\text{stale}} < \text{staleness} \leq \tau_{\text{off}}, \\ \text{ONLINE}, & \text{otherwise;} \end{cases} \quad (5)$$

$$A_{\text{stale}} = \int_0^T n_{\text{stale}}(t) dt, \quad A_{\text{off}} = \int_0^T n_{\text{off}}(t) dt, \quad (6)$$

with $\tau_{\text{stale}} = 10$ s and $\tau_{\text{off}} = 60$ s by default. Synchronisation delay is $t_{\text{delivered}} - t_{\text{generated}}$ per accepted message, reported as mean and 95th/99th percentiles. Equation (4) defines the staleness whose percentiles are reported throughout §6; (5) defines the three-state classification whose counts give the stale and

offline areas of (6), tabulated as `twin_stale_area_s` and `twin_offline_area_s` and reported per device so that they are comparable across fleet sizes. A *resynchronisation* is counted for each replayed telemetry message delivered after a disconnection, i.e. a reconnect recovery. Because the registry is write-only through delivery, all of these are measurements of the channel, not of the model’s self-report.

4.8. Determinism

A run is a pure function of (`SimConfig`, `seed`). `SeedPlan` derives independent `numpy.random.Generator` streams from a single `SeedSequence` for the job generator, thermal process noise, sensor noise, power noise, the fault scheduler, fault severities, connectivity, latency, loss, duplication and repair times. Spawning streams (rather than consuming one shared stream) means that adding a new stochastic source does not perturb the draws of the existing ones, which is what makes a scalability sweep and a fault sweep comparable across code changes.

5. Implementation and artifacts

The package is pure Python with NumPy at the core; `pandas`, `pyarrow`, `psutil`, `matplotlib`, `scipy`, `scikit-learn` and `paho-mqtt` are used by the experiment, analysis and adapter layers. Storage writers emit CSV, JSONL and Parquet, and every run writes a `metadata.json` carrying the full resolved configuration, the seed, the fault summary, the artifact manifest, the metrics, and a captured environment record (OS, machine, CPU count, total memory, Python implementation and version, and the pinned versions of every dependency). An optional MQTT adapter (`MqttTransport`, `MqttConfig`, `paho-mqtt` 2.1.0) publishes the same `Message` objects on the same topics through a real broker, using separate publisher and subscriber clients so that messages genuinely leave the process and return, and registering a retained MQTT Last Will on the `lwt` topic so the registry learns of ungraceful disconnects without polling. The adapter is selected by a single configuration flag (`mqtt.enabled`); the fleet engine never branches on the transport type, and the adapter reports the same counter names as the in-process transport, so every metric and table works over either. Because the simulated clock runs hundreds of times faster than the wall clock, an asynchronous transport always has a backlog at the end of a run; the transport interface therefore includes an explicit `flush` that drains it at the final simulated time, so the

Table 2: Baseline configuration shared by all experiments unless a subsection states otherwise.

Group	Parameter	Value
simulation	step Δt ; T_{tel} ; warmup	1 s; 1 000 ms; 60 s
thermal	setpoints (nozzle/bed/chamber)	210 / 60 / 45 °C
	C_i [J/K] (nozzle/bed/chamber)	16 / 1200 / 2700
	G_i [W/K] (nozzle/bed/chamber)	0.08 / 3.0 / 1.5
	P_i^{max} [W] (nozzle/bed/chamber)	40 / 250 / 150
	chamber coupling κ ; chamber heater	0.35; disabled
	K_p ; T^{int}	4.0 duty per error; 30 s
	process noise σ_ξ , τ_ξ ; sensor noise	0.15 K, 20 s; 0.25 K
power	standby; fans; motion; extruder	8 W; 3+5 W; 12 + 40 v/v_{nom} W; 8 W
material	filament $\emptyset$; layer; line; density	1.75 mm; 0.2 mm; 0.45 mm; 1.24 g cm ⁻³
health	wear rate; fault multiplier; repair; maint.	10 ⁻⁶ s ⁻¹ ; $\times 25$; 1800 s; 0.35
network	L_0 ; σ_j ; σ_{ln}	12 ms; 6 ms; 0.35
twin	τ_{stale} ; τ_{off}	10 s; 60 s
transport	replay queue limit; max message age	10 ⁴ messages; 300 s

tail of a run is attributed to latency rather than to loss. E7 (Section 6.7) exercises this adapter against Mosquitto 2.1.2. The unit-test suite contains 41 tests covering the thermal plant, the state machine, the fault injector, the transport, the MQTT adapter, the twin registry, determinism, storage round-trips and the metrics; the four MQTT tests skip themselves when no broker is listening.

6. Evaluation

Seven experiments, each stated as a question with a falsifiable expectation: §6.1 and §6.2 establish that the simulator is a credible instrument; §6.3 through §6.6 are the studies the instrument exists to make cheap; §6.7 checks the instrument against the real protocol. All experiments ran on a single Apple M3 Pro laptop (11 logical CPUs, 18 GiB RAM, Darwin 25.6.0, arm64), CPython 3.13.0, NumPy 2.5.3. No experiment uses more than one process. Table 2 gives the baseline parameters.

6.1. E1 — Scalability

Fleet size $n \in \{1, 10, 50, 100, 250, 500, 1000\}$, duration 3 600 s, $\Delta t = 1$ s, no faults, no impairment. Table 3 and Figure 1.

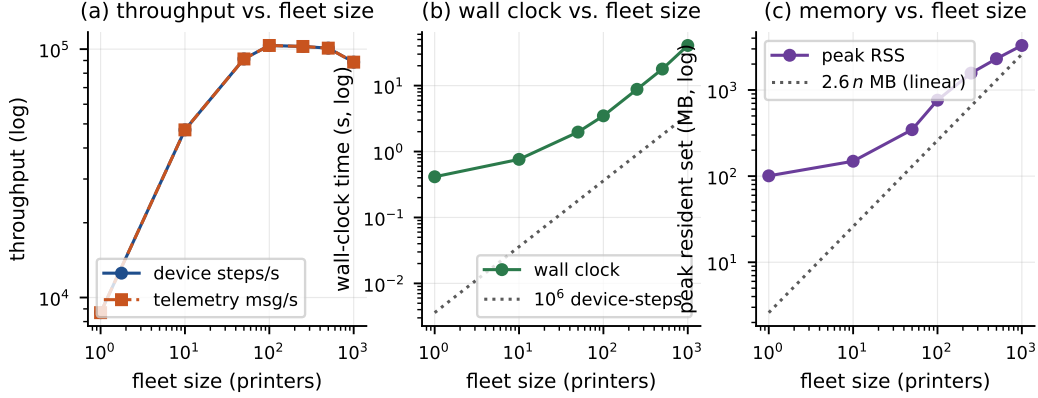


Figure 1: E1 scalability on a single 11-core laptop. (a) Device-steps/s and telemetry msg/s both peak near $n = 100$ and decline gently at $n = 1000$; (b) wall-clock time grows linearly in n (dotted reference: 10^6 device-steps); (c) peak resident set grows linearly at ≈ 3.3 MB per device (dotted reference: $3.3n$). All axes logarithmic.

Table 3: E1 scalability. Fleet size swept over three orders of magnitude at 3 600 s simulated duration, $\Delta t = 1$ s, on one core group of an 11-core laptop. RTF is the real-time factor.

Fleet n	Wall (s)	RTF	Device-steps s^{-1}	Telemetry (msg s^{-1})	Peak RSS (MB)	Generated	Delivery (%)
1	0.4142	8691	8691	8691	100.8	3600	99.97
10	0.7609	4731	4.731e+04	4.731e+04	148.9	3.6e+04	99.97
50	1.971	1826	9.13e+04	9.13e+04	346.4	1.8e+05	99.97
100	3.48	1035	1.035e+05	1.035e+05	762	3.6e+05	99.97
250	8.78	410	1.025e+05	1.025e+05	1570	9e+05	99.97
500	17.83	201.9	1.009e+05	1.009e+05	2308	1.8e+06	99.97
1000	40.61	88.64	8.864e+04	8.864e+04	3289	3.6e+06	99.97

Throughput peaks at $\approx 1.0 \times 10^5$ device-steps s^{-1} around $n = 100$ and falls to 8.86×10^4 at $n = 1000$; the real-time factor (simulated seconds per wall-clock second) is 8691 at $n = 1$ and 88.6 at $n = 1000$. The transition from a regime dominated by per-step fixed cost to one dominated by per-device array work is the expected signature of a vectorised engine, with the throughput decline beyond $n = 100$ consistent with cache and memory pressure: peak resident set grows from 101 MB at $n = 1$ to 3289 MB at $n = 1000$, i.e. ≈ 3.3 MB per device, on a machine with 18 GiB. Delivery ratio is flat at 99.97% across the sweep, confirming that the transport is not the scalability bottleneck at these sizes; the residual 0.03% is the initial step, before any telemetry has been generated.

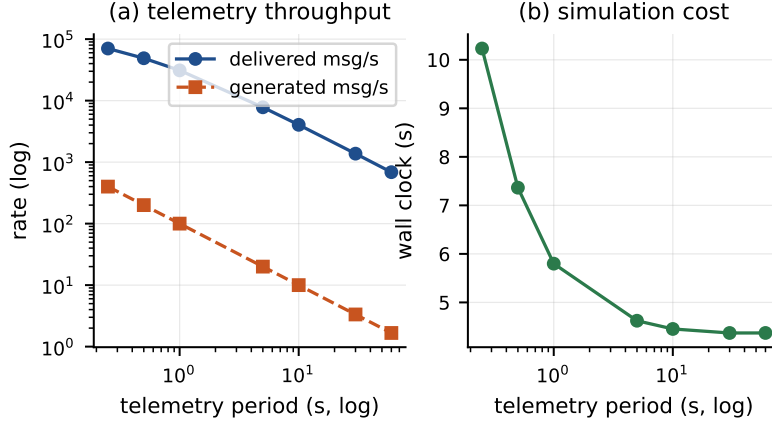


Figure 2: E2 telemetry frequency. (a) Delivered and generated message rates track $1/T_{\text{tel}}$ over 2.4 orders of magnitude; (b) the wall-clock cost of the same sweep changes by only a factor of 2.4, so the plant update, not the messaging, dominates.

Table 4: E2 telemetry frequency. $n = 100$, 1 800 s simulated, $\Delta t = 0.05$ s.

Period (ms)	Generated	Delivered	Telemetry (msg s ⁻¹)	Wall (s)	Peak RSS (MB)	Sync delay mean (ms)	Sync delay p95 (ms)
250	7.2e+05	7.2e+05	7.036e+04	10.23	1144	17.09	25.77
500	3.6e+05	3.6e+05	4.888e+04	7.365	1443	17.09	25.78
1000	1.8e+05	1.8e+05	3.104e+04	5.799	1454	17.1	25.81
5000	3.6e+04	3.6e+04	7788	4.622	900.9	17.12	25.87
1e+04	1.8e+04	1.8e+04	4042	4.453	772.9	17.15	25.98
3e+04	6000	6000	1373	4.37	735.6	17.15	26.04
6e+04	3000	3000	686.4	4.371	703.1	17.1	25.45

6.2. E2 — Telemetry frequency

$n = 100$, duration 1 800 s, $\Delta t = 0.05$ s (chosen so that every interval in the sweep is an integer multiple of the step), $T_{\text{tel}} \in \{250, 500, 1000, 5000, 10000, 30000, 60000\}$ ms. Table 4 and Figure 2.

Delivered throughput spans 686 to 70 363 msg s⁻¹, a factor of 102.5 across a 240-fold range of sampling periods; the shortfall from the ideal $1/T_{\text{tel}}$ scaling at the finest settings is the 0.03 % initial-step loss plus queueing at the sink. Wall-clock time grows only from 4.37 s to 10.23 s over the same 240-fold range, i.e. a factor of 2.3: message generation and delivery is *not* the dominant cost of the simulation, the per-step plant update is. Peak RSS, by contrast, scales with the number of stored messages (703 MB at 60 s to 1 144 MB at 250 ms), which is the practical constraint on high-rate telemetry in long runs, and the reason the writers stream to Parquet.

Table 5: E3 network impairment. Loss sweep (p varied) and outage sweep (mean interval between per-device disconnection episodes varied, mean episode duration 60 s). $n = 200$, 3 600 s.

Scenario	Loss p	Outage period (s)	Generated	Delivered	Delivery (%)	Latency mean (ms)	Latency max (ms)	p95 staleness (s)	Offline area (dev s)	Resyncs
loss	0	0	720000	719800	99.97	17.09	91.51	0.988	200	0
loss	0.01	0	720000	712663	98.98	17.09	91.51	1.988	203	0
loss	0.05	0	720000	684046	95.01	17.09	91.51	2.988	214	0
loss	0.1	0	720000	647915	89.99	17.09	91.51	3.987	220	0
loss	0.2	0	720000	575987	80	17.08	84.08	5.981	250	0
loss	0.3	0	720000	504368	70.05	17.08	84.08	6.987	274	0
outage	0	0	720000	719800	99.97	17.08	92.13	0.988	200	0
outage	0	1800	720000	719059	99.87	3647	1.88e+05	172	2.012e+04	40899
outage	0	900	720000	718714	99.82	6424	1.83e+05	175	3.5e+04	72598
outage	0	300	720000	716866	99.56	1.567e+04	1.88e+05	177	8.504e+04	180413
outage	0	120	720000	715172	99.33	2.847e+04	2.99e+05	179	1.528e+05	324942

6.3. E3 — Network impairment

$n = 200$, duration 3 600 s. Sweep 1 varies loss $p \in \{0, 0.01, 0.05, 0.10, 0.20, 0.30\}$. Sweep 2 varies the mean interval between per-device disconnection episodes, $\lambda_{\text{dev}}^{-1} \in \{\infty, 1800, 900, 300, 120\}$ s with mean episode duration 60 s. Table 5 and Figure 3.

Loss. Delivery ratio tracks the ideal $1 - p$ to within 0.05 percentage points at every setting, which is the correctness check for the loss model: 99.97 % at $p = 0$ down to 70.05 % at $p = 0.30$. Mean transport latency is flat at 17.1 ms, as expected for independent per-message loss. The informative signal is freshness: 95th-percentile twin staleness grows from 0.99 s at $p = 0$ to 6.99 s at $p = 0.30$, a slope of ≈ 20 s per unit loss $\approx T_{\text{tel}}/p$ in the small-loss limit—each lost telemetry sample pushes the twin’s age of information out by one sampling period, which is the AoI result for a lossy periodic source (Kaul et al., 2012) reproduced by construction. The binary stale/offline counters stay at zero because the configured $\tau_{\text{stale}} = 10$ s is not crossed by loss alone at these rates; Section 6.5 shows the regime where they are.

Outages. Delivery ratio barely moves (99.97 % $\rightarrow$ 99.33 %) because replay recovers the buffered messages, while the twin-side cost explodes: mean synchronisation delay rises from 17 ms to 28 471 ms, offline area from 200 to 152 798 device·s, and resynchronisations from 0 to 324 942. This is the central twin-synchronisation finding of the paper: *delivery* and *freshness* are different quantities, and a replay mechanism protects the first while doing nothing for the second. A supervisory application that reads the twin and sees “99.3 % of messages arrived” would conclude the link is healthy; an application that reads twin staleness sees a fleet that is on average 179 s out of date.

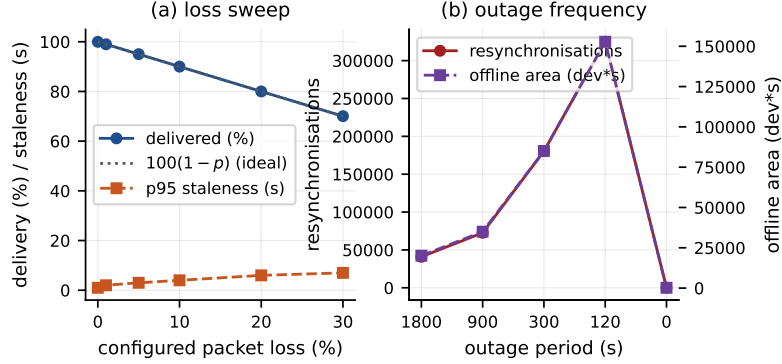


Figure 3: E3 network impairment. (a) Delivery ratio follows the ideal $1 - p$ while 95th-percentile twin staleness grows linearly in loss; (b) increasing outage frequency leaves delivery near-perfect while offline area and resynchronisation counts grow by five orders of magnitude.

6.4. E4 — Fault injection and separability

$n = 60$, duration 14 400 s (4 h), Poisson fault scheduling at 6 faults per machine-hour over the full catalogue, against a fault-free baseline at a different seed. 1 462 faults were injected across 11 types. For each fault type we compare telemetry samples recorded while the device was **PRINTING** with that fault active, against the fault-free **PRINTING** baseline, on seven channels: nozzle, bed and chamber temperature, power, print speed, cumulative material use, and health. We report Cohen’s d per channel and the Mann–Whitney AUC. Table 6 and Figure 4.

Every fault family separates from healthy printing on at least one channel with $|d| \geq 1.45$, which is “large” by Cohen’s conventions, and the separating channel is the mechanistically correct one. Bed-heater failure separates on *power* ($|d| = 3.68$) because the controller saturates duty against a heater that no longer delivers heat. Motor overheating ($|d| = 1.80$) and power anomaly ($|d| = 1.65$) separate on power; nozzle-heater failure separates on nozzle temperature ($d = -1.5$) with a compensating power rise; abnormal cooling separates on nozzle temperature ($d = -1.4$). The observational faults separate on *health* rather than on temperature mean, which is the signature of a sensing fault: the plant is untouched, so bed and chamber means barely move ($|d| \leq 0.23$) while the wear accumulator, driven by fault severity, moves the health channel ($|d| = 1.85$ for sensor noise, $|d| = 1.52$ for sensor drift).

The mean AUC across all seven channels sits at 0.42–0.50, i.e. near chance. That is not a failure of the model; it is the expected consequence

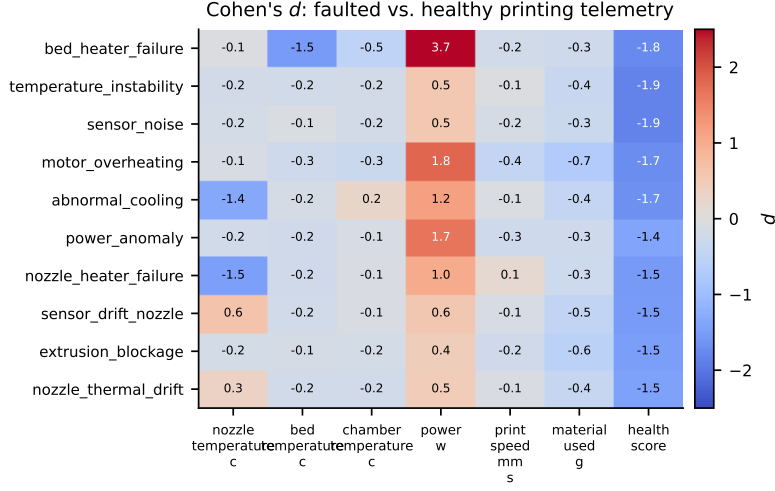


Figure 4: E4 fault separability. Per-channel Cohen’s d between faulted and healthy printing telemetry, ordered by largest absolute effect. The structure is mechanistic: thermal-actuator faults move temperature and power, power-domain faults move power, and observational faults move health while leaving temperature means near-unchanged.

of averaging a rank statistic over channels that a given fault does not touch, and it is the reason we report per-channel effect sizes. A detector that pools all channels will find nothing; a detector that looks at the mechanistically relevant channel finds a large, consistent separation. We read this as a concrete argument for mechanism-grounded feature selection in AM fault detection, and for publishing per-channel effect sizes rather than pooled scores.

6.5. E5 — Outage and recovery

$n = 50$, duration 3 600 s, per-device disconnection episodes with mean interval 300 s and mean duration 60 s, with exponential reconnect backoff and replay queues. 235 disconnection and 218 reconnection events were recorded, producing 45 131 resynchronisations and 40 414 device-s of offline area. Median (over fleet samples) worst-twin staleness was 234 s with a maximum of 302 s, and mean synchronisation delay 27 982 ms with a maximum of 299 014 ms—the latter is the replay-queue age limit interacting with the backoff. Delivery ratio was 98.84 %.

The recovery behaviour is the interesting part. Because a reconnecting device sends a retained full-state snapshot before resuming telemetry, the twin converges to the device within one accepted message after each reconnection: the offline area is finite and the staleness distribution has a ceiling set

Table 6: E4 fault separability. Per-channel Cohen’s d between telemetry recorded while the fault was active during PRINTING and the fault-free PRINTING baseline. Positive d means the fault raises the channel.

Fault	Samples	Max $ d $	ΔT_{nozzle}	ΔT_{bed}	ΔP (W)	Δv (mm/s)	Δm (g)	Δ health
bed_heater_failure	58941	3.68	-0.06	-1.53	3.68	-0.21	-0.27	-1.76
temperature_instability	60025	1.85	-0.18	-0.21	0.53	-0.05	-0.36	-1.85
sensor_noise	48584	1.85	-0.16	-0.06	0.54	-0.15	-0.32	-1.85
motor_overheating	50587	1.80	-0.13	-0.26	1.80	-0.44	-0.72	-1.73
abnormal_cooling	45099	1.69	-1.35	-0.15	1.15	-0.06	-0.44	-1.69
power_anomaly	44926	1.65	-0.20	-0.24	1.65	-0.26	-0.26	-1.40
nozzle_heater_failure	44207	1.54	-1.47	-0.25	1.00	0.14	-0.28	-1.54
sensor_drift_nozzle	68701	1.52	0.63	-0.23	0.55	-0.13	-0.45	-1.52
extrusion_blockage	54696	1.49	-0.17	-0.15	0.43	-0.24	-0.55	-1.49
nozzle_thermal_drift	47213	1.45	0.33	-0.20	0.46	-0.09	-0.41	-1.45

Table 7: E5 outage and recovery. $n = 50$, 3 600 s, mean disconnection interval 300 s, mean duration 60 s.

Fleet n	Simulated (s)	Delivery (%)	Latency mean (ms)	Latency max (ms)	p50 staleness (s)	Max staleness (s)	Stale area (devs)	Offline area (devs)	Resyncs	Seed
50	3600	98.84	2.798e+04	2.99e+05	234	302	5860	4.041e+04	4.513e+04	505
50	3600	98.84	2.798e+04	2.99e+05	234	302	5860	4.041e+04	4.513e+04	505

by the episode duration plus backoff, not by the number of episodes. Without the snapshot, replay of a 10^4 -message queue at 1 Hz would take longer than the run. This is the twin-side analogue of the last-will-and-testament plus retained-message pattern in MQTT 5 (OASIS Standard, 2019), and it is measurable here because the registry distinguishes *delivered* from *generated*. Table 7 gives the episode counts and the twin-side aggregates.

6.6. E6 — Reproducibility

Two runs at seed 2027 and one run at seed 2028, identical in every other respect. The two seed-2027 runs produce telemetry with identical SHA-256 digests (9e797bd1...965455); the seed-2028 run produces a different digest (995f11e1...b9a4cbd). Table 8. This is a strong form of reproducibility: bit-level identity of the artifact, not agreement of summary statistics to within a tolerance.

6.7. E7 — Broker fidelity

Everything reported so far is produced by the in-process transport, so a natural question is whether that transport is an abstraction over a real one or a simulation of one. E7 answers this by running the same 20-device fleet, the same topic scheme, the same `Message` objects and the same QoS

Table 8: E6 reproducibility. SHA-256 of the telemetry artifact. Two runs at the same seed are byte-identical; changing the seed changes the artifact.

Run	Telemetry SHA-256
seed 2027 run 1	9e797bd1f050c6ff...03965455
seed 2027 run 2	9e797bd1f050c6ff...03965455
seed 2028 run 1	995f11e1b52ff1b7...7b9a4cbd
<i>Same seed identical</i>	True
<i>Different seed differs</i>	True

settings through a real Mosquitto 2.1.2 broker on loopback, using the adapter of Section 5. Table 9.

Three things are worth reading off the table. First, **QoS 1 keeps the twin fresh and QoS 0 does not**: at QoS 1 the mean observed delivery latency is 3.8ms and twin staleness stays close to its telemetry-period floor at 1.2s, while at QoS 0 the mean latency is 51,594ms and p95 staleness rises to 15.1s, a 12.6-fold degradation. The mechanism is back-pressure: QoS 1 blocks the publisher on PUBACK, which throttles the producer to the broker’s service rate, whereas QoS 0 is fire-and-forget, so a producer running hundreds of times faster than real time fills the client outbox and the twin observes a queue it cannot drain. This is the same phenomenon that E3 isolates parametrically, reproduced on real infrastructure.

Second, **the broker path is lossless at both QoS levels**, including at a configured loss of 0.30, because the underlying transport is TCP: the configured loss is an additive impairment the in-process model applies, not a property of the broker. The two transports therefore answer different questions — the in-process transport answers “what does a given impairment budget do to twin freshness?” and the broker transport answers “what does this broker actually do?” E3 and E7 should be read together, not averaged.

Third, **the broker path is slower but not prohibitive**: the QoS 1 broker run sustains an RTF of 156 against 445 for the in-process transport, i.e. a real broker round trip costs roughly two thirds of the headroom at this fleet size. Running a 1000-device study over a broker is feasible; running the full E1 sweep over a broker is not, which is why the scalability results use the in-process transport.

Table 9: E7 broker fidelity. The same 20-device fleet, the same topics and the same message objects, run through a real Mosquitto 2.1.2 broker on loopback and through the in-process transport. QoS 1 blocks on PUBACK, which applies back-pressure and keeps the twin fresh; QoS 0 is fire-and-forget, so the simulated clock outruns the broker and staleness grows by an order of magnitude. The broker path is lossless over TCP at both QoS levels: the configured loss is an additive impairment the in-process model applies, not a property of the broker.

Transport	QoS	Cfg. loss	Gen.	Deliv.	Deliv. (%)	Lat. mean (ms)	Lat. max (ms)	Stale p95 (s)	Stale max (s)
Broker	0	0.00	2400	2400	100.00	51,594.5	101,950.0	15.10	18.05
Broker	0	0.30	2400	2400	100.00	48,892.5	93,950.0	11.65	15.65
Broker	1	0.00	2400	2400	100.00	3.8	1,000.0	1.20	1.95
Broker	1	0.30	2400	2400	100.00	2.1	1,000.0	0.95	1.95
In-process	0	0.00	2400	2400	100.00	0.0	0.0	0.90	0.95
In-process	0	0.30	2400	1703	70.96	0.0	0.0	4.95	7.95
In-process	1	0.00	2400	2400	100.00	0.0	0.0	0.90	0.95
In-process	1	0.30	2400	1703	70.96	0.0	0.0	4.95	7.95

7. Dataset package and reproducibility

Each run directory contains `telemetry.csv`, `telemetry.parquet`, `events.csv`, `twins.parquet` (where written) and `metadata.json`. The metadata records the resolved configuration, the seed, the fault summary, the artifact manifest with per-file row counts, the run metrics, and the captured environment: OS release, machine and processor, logical CPU count, total memory, Python implementation and version, and the pinned versions of all ten dependencies (NumPy 2.5.3, pandas 3.0.6, pyarrow 25.0.1, psutil 7.2.2, matplotlib 3.11.2, scipy 1.18.1, scikit-learn 1.9.1, PyYAML 6.0.3, paho-mqtt 2.1.0, pytest 9.1.1). Per-experiment `summary.csv` files aggregate the sweeps, and `results/bibliography_verification.{json,csv}` records the Crossref resolution of every bibliography entry.

Reporting guidelines for simulation-based research (Rahmandad and Sterman, 2012) motivate three specific choices: an explicit warmup window excluded from analysis (60s), a single seed per sweep point with the seed recorded in every summary row, and publication of the generating code alongside the data, so that a reader can regenerate any figure from the repository root with `experiments/run_all.py` followed by `experiments/make_figures.py`.

8. Threats to validity and limitations

Model fidelity. The thermal plant is a three-node lumped model with constant coefficients. It reproduces the qualitative behaviour a twin must track—setpoint tracking, first-order rise, saturating duty, the effect of a parasitic heat load—and it is calibrated to the *order of magnitude* of desktop FDM hardware, not fitted to any specific machine. It does not model melt-pool dynamics, layer-to-layer heat accumulation with temperature-dependent conductivity, convection that varies with part geometry, filament moisture, or crystallinity. Claims about bead geometry, residual stress, warping, or part strength are out of scope, and SimTwin should not be used for them. The deposition model is a road-geometry volume integral; it has no path planner.

Network fidelity. The transport models delivery semantics (latency, jitter, loss, duplication, disconnection, outage, replay, staleness) as independent stochastic mechanisms. It does not model bandwidth, contention, TCP congestion windows, TLS handshake cost, broker CPU, queueing at the broker, or the network stack of a real device. The reported msg/s figures are therefore the simulator’s own generation and delivery cost, not a broker’s capacity; broker capacity is measured against real brokers elsewhere (Mishra et al., 2021; Silva et al., 2021). The MQTT adapter lets a user replace the in-process bus with a real broker, and E7 exercises it, but only at $n = 20$: a broker round trip costs roughly two thirds of the simulator’s headroom (RTF 150 against 449), so the 1000-device sweep in E1 could not be run over a broker in reasonable time. The scalability and fault results are therefore properties of the in-process transport, and E7 is the only result measured against real infrastructure.

Fault realism. Faults are parameter mechanisms with severity ranges drawn from engineering judgement, not from failure-mode data fitted to a fleet. The separability result shows that the injected mechanisms produce signatures that are large and mechanistically coherent; it does not show that real FDM faults have those magnitudes. The 11-fault catalogue is a starting point, not a taxonomy of FDM failure. One catalogue fault (`print_interruption`) is terminal and short-lived, so it produced fewer than 30 qualifying `PRINTING` samples and is absent from Table 6; its effect is on job outcomes, not on telemetry distributions.

Statistics. Each sweep point is a single seeded run, not a set of replicates, so the sweeps carry no confidence intervals. The determinism result (E6)

makes this a deliberate, reproducible choice rather than an accident, and the separability statistics in E4 are computed over 4.4×10^4 – 6.9×10^4 samples per fault, which is ample for the rank-based statistics used. Effect sizes, not hypothesis tests, are the reported quantity.

Timing variance. The simulated physics is exactly reproducible; the *wall-clock* columns are not. They are single measurements on a general purpose laptop, and re-running the same configuration on the same machine moves them by ten to twenty per cent with background load — the $n = 1000$ E1 point has been observed at both 70 611 and 88 643 device-steps s^{-1} , and the E7 QoS 0 staleness figure at both 13.2 s and 15.1 s, on identical configurations and seeds. The reported figures are from the final full-suite run, whose generating code, configuration and seeds are all published. Scalability *trends* — the peak near $n = 100$, the linear RSS growth, the monotone staleness response — are stable across runs; the absolute constants are indicative, not certified, and any comparison against them should be made on the same hardware under comparable load.

Scope of the evidence. The largest fleet is 1 000 devices for one simulated hour. We do not claim behaviour at 10^5 devices, over multi-day horizons, or under the memory pressure that a long high-rate telemetry run on a smaller machine would produce.

Bibliography. Every entry in this paper was resolved against the Crossref REST API. Five candidate sources—a networked-control survey, a predictive-maintenance dataset paper, and three AM-energy papers—could not be resolved to the works described at any DOI we could verify, and were *dropped* rather than cited with an unverifiable identifier. Several further entries had a resolvable DOI but a year, first author or journal attribution that Crossref contradicted, and were corrected to the resolved record (Baumers et al., 2017; Liu et al., 2023; Grieves and Vickers, 2016). The machine-readable audit is published with the artifacts.

9. Conclusion

The three findings of §6—that delivery and freshness diverge, that fault signal is per-channel, and that determinism is a property of the composition rather than of any one module—are the reason this simulator is worth building on.

We presented SimTwin, an open-source, deterministic, mechanism-level simulator of networked FDM/FFF fleets that couples a physical plant, a job

and health state machine, a message-level transport with explicit delivery semantics, and a twin registry whose state is derived only from delivered messages. The evaluation establishes that the tool runs at useful scale on commodity hardware, that its channel model reproduces the analytic loss and age-of-information behaviour it is meant to represent, that delivery and freshness are measurably different quantities under outage, and that injected fault mechanisms produce large, mechanistically coherent, per-channel separability from healthy operation. The artifact—source, tests, experiment scripts, per-run metadata, summary tables, figures and a Crossref-verified bibliography—is published as a package, and every figure in this paper can be regenerated from the repository root by two commands.

Future work follows directly from the limitations: fitting the thermal and fault parameters against instrumented machines; coupling the transport to a real broker and measuring the delta; adding replicates and reporting confidence intervals; extending the fault catalogue from failure-mode data; and using the twin synchronisation metrics as an explicit objective in a scheduling or edge-offloading study.

Data and code availability

The SimTwin package, the experiment scripts (`experiments/run_all.py`, `experiments/make_figures.py`), the unit-test suite, the per-run artifacts, the summary tables, the figures and the bibliography verification audit are published together in the project repository. Running `pythonexperiments/run_all.py` regenerates every result, and `pythonexperiments/make_figures.py` regenerates every figure and every table in this paper.

CRedit author statement

N.S.: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Writing—original draft, Writing—review & editing, Visualization.

Declaration of competing interest

The author reports financial interests in G3D, a 3D-printer and additive-manufacturing company, and in Versec LLC, an automation and technology-development studio. The author declares no competing financial interest in the results reported in this paper.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Generative-AI use statement

Machine-assisted tooling was used for literature retrieval, Crossref verification scripting, code scaffolding and drafting support. All scientific decisions, model design, experiment design, interpretation and the final bibliography audit are the author’s responsibility, and every citation was verified against Crossref as described in Section 8.

References

- Alaa Alatrani, Leslie F. Sikos, Mike Johnstone, Patryk Szewczyk, and James Jin Kang. DoS/DDoS-MQTT-IoT: A dataset for evaluating intrusions in IoT networks using the MQTT protocol. *Computer Networks*, 231:109809, 2023. doi: 10.1016/j.comnet.2023.109809.
- Turki Alhazmi, Farag Azzedin, and Mohammad Hammoudeh. MQTT Based Data Distribution Framework for Digital Twin Networks. In *Proceedings of the 8th International Conference on Future Networks & Distributed Systems*, pages 1008–1013. ACM, 2024. doi: 10.1145/3726122.3726269.
- Gabriel Aznar Lapuente, Paula Morella Avinzano, Maria Pilar Lamban Castillo, and Dammika Seneviratne. Methodologies in digital twin for manufacturing industry: A systematic literature review. *Future Generation Computer Systems*, 174:107997, 2026. doi: 10.1016/j.future.2025.107997.
- Barbara Rita Barricelli, Elena Casiraghi, and Daniela Fogli. A Survey on Digital Twin: Definitions, Characteristics, Applications, and Design Implications. *IEEE Access*, 7:167653–167671, 2019. doi: 10.1109/access.2019.2953499.
- Martin Baumer, Joost R. Duflou, William Flanagan, Timothy G. Gutowski, Karel Kellens, and Reid Lifset. Charting the Environmental Dimensions of Additive Manufacturing and 3D Printing. *Journal of Industrial Ecology*, 21(S1), 2017. doi: 10.1111/jiec.12668.

- Ane Blázquez-García, Angel Conde, Usue Mori, and Jose A. Lozano. A Review on Outlier/Anomaly Detection in Time Series Data. *ACM Computing Surveys*, 54(3):1–33, 2021. doi: 10.1145/3444690.
- Markus M. Breunig, Hans-Peter Kriegel, Raymond T. Ng, and Jörg Sander. LOF. In *Proceedings of the 2000 ACM SIGMOD international conference on Management of data*, pages 93–104. ACM, 2000. doi: 10.1145/342009.335388.
- S Cavalieri and G Cutuli. Performance evaluation of OPC UA. In *2010 IEEE 15th Conference on Emerging Technologies & Factory Automation (ETFA 2010)*, pages 1–8. IEEE, 2010. doi: 10.1109/ETFA.2010.5641184.
- Yechang Cho and Sang Do Noh. Design and Implementation of Digital Twin Factory Synchronized in Real-Time Using MQTT. *Machines*, 12(11):759, 2024. doi: 10.3390/machines12110759.
- Carlos Faria, Jaime Fonseca, and Estela Bicho. FIBR3DEmul—an open-access simulation solution for 3D printing processes of FDM machines with 3+ actuated axes. *The International Journal of Advanced Manufacturing Technology*, 106(7-8):3609–3623, 2020. doi: 10.1007/s00170-019-04713-y.
- Aidan Fuller, Zhong Fan, Charles Day, and Chris Barlow. Digital Twin: Enabling Technologies, Challenges and Open Research. *IEEE Access*, 8: 108952–108971, 2020. doi: 10.1109/access.2020.2998358.
- Michael Grieves and John Vickers. Digital twin: Manufacturing excellence through virtual factory replication. White paper, Florida Institute of Technology, Melbourne, FL, USA, 2014. URL <https://www.3ds.com/fileadmin/PRODUCTS-SERVICES/DELMIA/PDF/Whitepaper/DELMIA-APRISO-Digital-Twin-Whitepaper.pdf>. White paper; no DOI assigned.
- Michael Grieves and John Vickers. Digital Twin: Mitigating Unpredictable, Undesirable Emergent Behavior in Complex Systems. In *Transdisciplinary Perspectives on Complex Systems*, pages 85–113. Springer International Publishing, 2016. doi: 10.1007/978-3-319-38756-7_4.
- Christopher M. Henson, Nathan I. Decker, and Qiang Huang. A digital twin strategy for major failure detection in fused deposition modeling processes. *Procedia Manufacturing*, 53:359–367, 2021. doi: 10.1016/j.promfg.2021.06.039.

- C. Human, A. H. Basson, and K. Kruger. Digital Twin Data Pipeline Using MQTT in SLADTA. In *Studies in Computational Intelligence*, pages 111–122. Springer International Publishing, 2021. doi: 10.1007/978-3-030-69373-2_7.
- Sergio Infante, Julia Robles, Cristian Martín, Bartolomé Rubio, and Manuel Díaz. Distributed digital twins on the open-source OpenTwins framework. *Advanced Engineering Informatics*, 64:102970, 2025. doi: 10.1016/j.aei.2024.102970.
- International Organization for Standardization. Automation systems and integration — digital twin framework for manufacturing: Part 1, overview and general principles. ISO 23247-1:2021, 2021. URL <https://www.iso.org/standard/75066.html>. Standard; no DOI assigned.
- International Organization for Standardization. Information technology — digital twin — concepts and terminology. ISO/IEC 30173:2023, 2023. URL <https://www.iso.org/standard/81442.html>. Standard; no DOI assigned.
- Rolf Isermann. Model-based fault-detection and diagnosis – status and applications. *Annual Reviews in Control*, 29(1):71–85, 2005. doi: 10.1016/j.arcontrol.2004.12.002.
- Michael Jacoby and Thomas Usländer. Digital Twin and Internet of Things—Current Standards Landscape. *Applied Sciences*, 10(18):6519, 2020. doi: 10.3390/app10186519.
- Nursultan Jyeniskhan, Kemel Shomenov, Md Hazrat Ali, and Essam Shehab. Exploring the integration of digital twin and additive manufacturing technologies. *International Journal of Lightweight Materials and Manufacture*, 7(6):860–881, 2024. doi: 10.1016/j.ijlmm.2024.06.004.
- Sanjit Kaul, Roy Yates, and Marco Gruteser. Real-time status: How often should one update? In *2012 Proceedings IEEE INFOCOM*, pages 2731–2735. IEEE, 2012. doi: 10.1109/INFCOM.2012.6195689.
- Sujitha Lakshminarayana, Amit Praseed, and P. Santhi Thilagam. Securing the IoT Application Layer From an MQTT Protocol Perspective: Challenges and Research Prospects. *IEEE Communications Surveys & Tutorials*, 26(4):2510–2546, 2024. doi: 10.1109/COMST.2024.3372630.

- Lars Larsen, Thomas Fraunholz, Tim Köhler, Dennis Rall, Veronika Langner, Dominik Görick, and Alfons Schuster. RECAST: An Open-Source Digital Twin Framework for Industrial Production Environments. In *Lecture Notes in Mechanical Engineering*, pages 160–171. Springer Nature Switzerland, 2024. doi: 10.1007/978-3-031-74482-2_19.
- Fei Tony Liu, Kai Ming Ting, and Zhi-Hua Zhou. Isolation Forest. In *2008 Eighth IEEE International Conference on Data Mining*, pages 413–422. IEEE, 2008. doi: 10.1109/ICDM.2008.17.
- Shimin Liu, Pai Zheng, and Jinsong Bao. Digital Twin-based manufacturing system: a survey based on a novel reference model. *Journal of Intelligent Manufacturing*, 35(6):2517–2546, 2023. doi: 10.1007/s10845-023-02172-7.
- Biswajeeban Mishra and Attila Kertesz. The Use of MQTT in M2M and IoT Systems: A Survey. *IEEE Access*, 8:201071–201086, 2020. doi: 10.1109/ACCESS.2020.3035849.
- Biswajeeban Mishra, Biswaranjan Mishra, and Attila Kertesz. Stress-Testing MQTT Brokers: A Comparative Analysis of Performance Measurements. *Energies*, 14(18):5817, 2021. doi: 10.3390/en14185817.
- Michael Nast, Hannes Raddatz, Benjamin Rother, Frank Golasowski, and Dirk Timmermann. A Survey and Comparison of Publish/Subscribe Protocols for the Industrial Internet of Things (IIoT). In *Proceedings of the 12th International Conference on the Internet of Things*, pages 193–200. ACM, 2022. doi: 10.1145/3567445.3571107.
- Walter Nieminen, Haben Gebreweld, Arto Liuha, Mikko Nissinen, Mario Verdugo, Aki Mikkola, and Antero Kutvonen. Synthetic Data for Predictive Maintenance: A Systematic Review and Framework for Industry 4.0 Applications. *Journal of Intelligent Manufacturing*, 2026. doi: 10.1007/s10845-026-02795-6.
- OASIS Standard. Mqtt version 3.1.1. OASIS Standard, 29 October 2014, 2014. URL <http://docs.oasis-open.org/mqtt/mqtt/v3.1.1/os/mqtt-v3.1.1-os.html>. Industry specification; no DOI assigned.
- OASIS Standard. Mqtt version 5.0. OASIS Standard, 3 December 2019, 2019. URL <https://docs.oasis-open.org/mqtt/mqtt/v5.0/os/mqtt-v5.0-os.html>. Industry specification; no DOI assigned.

- Minas Pantelidakis, Konstantinos Mykoniatis, Jia Liu, and Gregory Harris. A digital twin ecosystem for additive manufacturing using a real-time development platform. *The International Journal of Advanced Manufacturing Technology*, 120(9-10):6547–6563, 2022. doi: 10.1007/s00170-022-09164-6.
- Hazhir Rahmandad and John D. Sterman. Reporting guidelines for simulation-based research in social sciences. *System Dynamics Review*, 28(4): 396–411, 2012. doi: 10.1002/sdr.1481.
- Julia Robles, Cristian Martín, and Manuel Díaz. OpenTwins: An open-source framework for the development of next-gen compositional digital twins. *Computers in Industry*, 152:104007, 2023. doi: 10.1016/j.compind.2023.104007.
- Christian Schwede and Daniel Fischer. Learning Simulation-Based Digital Twins for Discrete Material Flow Systems: A Review. In *2024 Winter Simulation Conference (WSC)*, pages 3070–3081. IEEE, 2024. doi: 10.1109/WSC63780.2024.10838729.
- Bernhard Schölkopf, John C. Platt, John Shawe-Taylor, Alex J. Smola, and Robert C. Williamson. Estimating the Support of a High-Dimensional Distribution. *Neural Computation*, 13(7):1443–1471, 2001. doi: 10.1162/089976601750264965.
- Victor Seoane, Carlos Garcia-Rubio, Florina Almenares, and Celeste Campo. Performance evaluation of CoAP and MQTT with security support for IoT environments. *Computer Networks*, 197:108338, 2021. doi: 10.1016/j.comnet.2021.108338.
- Daniel Silva, Liliana I. Carvalho, José Soares, and Rute C. Sofia. A Performance Analysis of Internet of Things Networking Protocols: Evaluating MQTT, CoAP, OPC UA. *Applied Sciences*, 11(11):4879, 2021. doi: 10.3390/app11114879.
- Tomasz Szydło, Joanna Sendorek, Mateusz Windak, and Robert Brzoz-Woch. Dataset for Anomalies Detection in 3D Printing. In *Lecture Notes in Computer Science*, pages 647–653. Springer International Publishing, 2021. doi: 10.1007/978-3-030-77970-2_50.

- Barış Tan and Andrea Matta. The digital twin synchronization problem: Framework, formulations, and analysis. *IIEE Transactions*, 56(6):652–665, 2023. doi: 10.1080/24725854.2023.2253869.
- Fei Tao, Jiangfeng Cheng, Qinglin Qi, Meng Zhang, He Zhang, and Fangyuan Sui. Digital twin-driven product design, manufacturing and service with big data. *The International Journal of Advanced Manufacturing Technology*, 94(9-12):3563–3576, 2017. doi: 10.1007/s00170-017-0233-1.
- Fei Tao, He Zhang, Ang Liu, and A. Y. C. Nee. Digital Twin in Industry: State-of-the-Art. *IEEE Transactions on Industrial Informatics*, 15(4):2405–2415, 2019. doi: 10.1109/TII.2018.2873186.
- Jan-Frederik Uhlenkamp, Jannicke Baalsrud Hauge, Eike Broda, Michael Lutjen, Michael Freitag, and Klaus-Dieter Thoben. Digital Twins: A Maturity Model for Their Classification and Evaluation. *IEEE Access*, 10:69605–69635, 2022. doi: 10.1109/access.2022.3186353.
- G.C. Walsh, Hong Ye, and L.G. Bushnell. Stability analysis of networked control systems. *IEEE Transactions on Control Systems Technology*, 10(3):438–446, 2002. doi: 10.1109/87.998034.
- A. Wippermann, Timothy G. Gutowski, B. Denkena, M.-A. Dittrich, and Y. Wessargues. Electrical energy and material efficiency analysis of machining, additive and hybrid manufacturing. *Journal of Cleaner Production*, 251:119731, 2020. doi: 10.1016/j.jclepro.2019.119731.
- Roy D. Yates, Yin Sun, D. Richard Brown, Sanjit K. Kaul, Eytan Modiano, and Sennur Ulukus. Age of Information: An Introduction and Survey. *IEEE Journal on Selected Areas in Communications*, 39(5):1183–1210, 2021. doi: 10.1109/JSAC.2021.3065072.
- Zahra Zamanzadeh Darban, Geoffrey I. Webb, Shirui Pan, Charu Aggarwal, and Mahsa Salehi. Deep Learning for Time Series Anomaly Detection: A Survey. *ACM Computing Surveys*, 57(1):1–42, 2024. doi: 10.1145/3691338.
- Wei Zhang, Michael S. Branicky, and Stephen M. Phillips. Stability of networked control systems. *IEEE Control Systems Magazine*, 21(1):84–99, 2001. doi: 10.1109/37.898794.